贪心算法综合题题库（理论分析型）

题型说明

贪心算法综合题的典型作答要求：

1. **分析贪心选择性质**：证明局部最优能导致全局最优
2. **分析最优子结构**：证明问题具有最优子结构
3. **设计贪心策略**：给出贪心选择的具体规则
4. **伪代码实现**：描述算法步骤
5. **复杂度分析**：通常为排序+遍历 O(n log n)

题目0：找零钱问题（贪心入门案例）

【问题描述】 假设有面值为$c_1 > c_2 > \dots > c_k$的硬币系统，且$c_k = 1$（保证任意金额都能凑出）。设计贪心算法找出凑成金额M所需的最少硬币数。

【输入示例】 硬币面额：[100, 50, 20, 10, 5, 1]，金额 M = 186

【输出示例】 最少硬币数：6枚（100+50+20+10+5+1）

【答题框架】

1. 贪心策略分析

贪心策略：每次选择不超过剩余金额的最大面额硬币。

贪心选择性质：对于规范币制（如1,5,10,20,50,100），贪心选择能导致全局最优解。但对于某些币制（如1,3,4），M=6时贪心得4+1+1=3枚，但最优解为3+3=2枚，贪心失效。

证明思路（规范币制下）：设最大面额为c_1，对于任意金额M，若使用k枚c_1，则剩余金额小于c_1。可以证明任何最优解中c_1的使用数量与贪心解相同，否则会产生更小面额硬币的组合，其总价值与c_1等价但数量更多。

2. 伪代码

text

```
算法 CoinChange_greedy(coins, M)
// 输入：硬币面额数组coins（降序排列），金额M
// 输出：所需硬币数

1. count ← 0
```

```
2.  remaining ← M
3.
4.  for i ← 0 to len(coins)-1 do
5.      if remaining = 0 then
6.          break
7.      end if
8.      use ← remaining // coins[i]    // 整除，计算用多少枚当前面额
9.      count ← count + use
10.     remaining ← remaining - use * coins[i]
11. end for
12.
13. return count
```

3. 时间复杂度分析

- 硬币种类数为k，循环遍历一次
- 总时间复杂度：**O(k)**
- 空间复杂度：**O(1)**

题目1：活动选择问题（区间调度）

【问题描述】 有n个活动，每个活动i有一个开始时间s[i]和结束时间f[i]（s[i] < f[i]）。活动之间不能重叠（端点接触也算不重叠）。请选择尽可能多的互不重叠的活动。

【输入示例】（按结束时间排序后）
活动：1:(1,4), 2:(3,5), 3:(0,6), 4:(5,7), 5:(3,9), 6:(5,9), 7:(6,10), 8:(8,11), 9:(8,12), 10:(2,14)

【输出示例】 最多可选4个活动，如(1,4)、(5,7)、(8,11)等

【答题框架】

1. 贪心策略分析

贪心策略：每次选择**结束时间最早**且与已选活动不冲突的活动。

贪心选择性质证明：

设活动集合S，按结束时间升序排列为$a_1, a_2, \ldots, a_n$。令a_k为结束时间最早的活动。

要证明：存在一个最优解包含a_k。

- 设A是一个最优解，其中结束时间最早的活动是a_j（$j \geq 1$）
- 若$j = k$，则结论成立
- 若$j > k$，则a_k的结束时间更早，且a_k与A中除a_j外的其他活动也不冲突（因为a_k结束时间更早）
- 构造$A' = (A \setminus \{a_j\}) \cup \{a_k\}$，则A'也是可行解，且$|A'| = |A|$

- 所以A'也是一个最优解，且包含a_k

因此，贪心选择不会丢失最优解。✅

最优子结构：选择了a_k后，原问题变为在开始时间 ≥ f_k的活动集合中选择最多活动，是原问题的子问题。

2. 伪代码

text

```
算法 ActivitySelection(s, f, n)
// 输入：开始时间数组s[1..n]，结束时间数组f[1..n]，已按结束时间升序排序
// 输出：选中的活动列表

1.  A ← {1}                  // 选择第一个活动（结束最早）
2.  lastEnd ← f[1]
3.
4.  for i ← 2 to n do
5.      if s[i] ≥ lastEnd then
6.          A ← A ∪ {i}
7.          lastEnd ← f[i]
8.      end if
9.  end for
10.
11. return A
```

3. 时间复杂度分析

- 若活动已按结束时间排序：O(n)
- 若未排序：需要先排序 O(n log n)
- 总时间复杂度：**O(n log n)**
- 空间复杂度：**O(1)**（不计存储输入）

题目2：哈夫曼编码

【问题描述】 给定一组字符及其出现频率（或权重），设计一种前缀码（没有任何码字是其他码字的前缀），使得编码后的总长度最小。

【输入示例】 字符频率：A:45, B:13, C:12, D:16, E:9, F:5

【输出示例】 最优二叉树及各字符的编码（如A:0, B:101, C:100, D:111, E:1101, F:1100）

【答题框架】

1. 贪心策略分析

贪心策略：反复选择**频率最低**的两个节点合并，生成一个新节点（频率为两者之和），直到只剩一个节点（根节点）。

贪心选择性质证明：

设x和y是频率最低的两个字符。要证明存在一个最优编码树中，x和y是兄弟节点且深度最大。

- 设T是一个最优编码树，x和y在其中。若x和y不是兄弟，则存在两个深度更大的兄弟节点a和b
- 交换x与a、y与b后，新的树T'的加权路径长度更小（因为将低频率节点放在更深的位置，高频率节点放在更浅的位置，总代价降低）
- 因此最优树中，x和y必须是兄弟，且处于最深一层

最优子结构：合并x和y为新节点z（频率f(x)+f(y)）后，原问题转化为在字符集(S \ {x,y}) ∪ {z}上构造最优编码树，这是一个子问题。

2. 伪代码

text

```
算法 Huffman(C)
// 输入：字符集合C，每个字符c有频率f[c]
// 输出：哈夫曼树的根节点

1.  Q ← 优先队列（最小堆），初始包含C中所有节点
2.
3.  for i ← 1 to |C|-1 do
4.      x ← ExtractMin(Q)      // 取出频率最小的节点
5.      y ← ExtractMin(Q)      // 取出频率次小的节点
6.      z ← NewNode()          // 创建新节点
7.      z.freq ← x.freq + y.freq
8.      z.left ← x
9.      z.right ← y
10.     Insert(Q, z)           // 将新节点插入队列
11. end for
12.
13. return ExtractMin(Q)       // 返回根节点
```

3. 时间复杂度分析

- 用二叉堆实现优先队列：每次ExtractMin和Insert为O(log n)
- 循环执行n-1次
- 总时间复杂度：**O(n log n)**
- 空间复杂度：**O(n)**

题目3：Prim算法——最小生成树

【问题描述】 给定一个带权连通无向图G = (V, E)，其中V为顶点集，E为边集，每条边带有权重w(u, v)。求一棵生成树T ⊆ E，使得T连接所有顶点且边权之和最小。

【输入示例】 图结构（邻接矩阵或边列表），例如：

text

```
顶点数 n = 5
边列表：(1,2,2), (1,3,4), (2,3,1), (2,4,3), (3,4,5), (3,5,6), (4,5,7)
```

【输出示例】 最小生成树总权重：12，边集：{(2,3,1), (1,2,2), (2,4,3), (3,5,6)} 或等价形式

【答题框架】

1. 贪心策略分析

贪心策略（Prim算法）：从任意一个顶点开始，每次选择连接**已选顶点集合**与**未选顶点集合**之间的**最小权重边**，将新顶点加入集合，直到所有顶点都被覆盖。

贪心选择性质证明：

设已选顶点集为U，未选顶点集为V-U。令边(u, v)是连接U与V-U的最小权边，其中u∈U，v∈V-U。

要证明：存在一棵最小生成树包含边(u, v)。

- 设T是一棵最小生成树，若T不包含(u, v)，则T中存在一条连接U和V-U的边(u', v')（否则图不连通）
- 将(u, v)加入T会形成环，环中必然存在另一条连接U和V-U的边(u', v')，且w(u, v) ≤ w(u', v')
- 移除(u', v')得到新树T'，权值不大于T
- 因此T'也是一棵最小生成树，且包含(u, v)

因此，贪心选择不会丢失最优解。✅

最优子结构：选择边(u, v)后，原问题转化为在收缩边(u, v)后的图上求最小生成树，是原问题的子问题。

2. 伪代码（优先队列优化版）

text

```
算法 Prim(G, r)
// 输入：图G，起始顶点r
// 输出：最小生成树的边集和总权重

1.  for each vertex v in V do
2.      key[v] ← ∞          // 连接v到U的最小边权
3.      parent[v] ← NIL     // 记录父节点
4.  end for
```

```
5.  key[r] ← 0
6.  Q ← 优先队列（最小堆），包含所有顶点，按key值排序
7.  totalWeight ← 0
8.  edgeSet ← ∅
9.
10. while Q is not empty do
11.     u ← ExtractMin(Q)        // 取出key最小的顶点
12.     totalWeight ← totalWeight + key[u]
13.     if parent[u] ≠ NIL then
14.         edgeSet ← edgeSet ∪ {(parent[u], u)}
15.     end if
16.
17.     for each neighbor v of u do
18.         if v in Q and w(u, v) < key[v] then
19.             key[v] ← w(u, v)
20.             parent[v] ← u
21.             DecreaseKey(Q, v, key[v])   // 更新优先队列
22.         end if
23.     end for
24. end while
25.
26. return totalWeight, edgeSet
```

3. 时间复杂度分析

- 使用二叉堆实现优先队列：
  - ExtractMin操作：O(log V)，共V次 → O(V log V)
  - DecreaseKey操作：O(log V)，共E次 → O(E log V)
- 总时间复杂度：**O((V + E) log V)**
- 对于稠密图（E ≈ V²），可用斐波那契堆优化至 **O(E + V log V)**
- 空间复杂度：**O(V + E)**

题目4：Kruskal算法——最小生成树

【问题描述】 给定一个带权连通无向图G = (V, E)，求一棵最小生成树。

【输入示例】 同题目3

【输出示例】 同题目3

【答题框架】

1. 贪心策略分析

贪心策略（Kruskal算法）：将图中所有边按权重**从小到大**排序，依次检查每条边，若该边连接的两个顶点属于不同的连通分量，则将其加入生成树（否则会形成环），直到生成树包含V-1条边。

贪心选择性质证明：

设边列表按权重升序为$e_1, e_2, ..., e_m$。令e_k是第一条连接两个不同连通分量的边（即不形成环的边）。

要证明：存在一棵最小生成树包含e_k。

- 设T是一棵最小生成树，若T不包含e_k，则e_k连接的两个顶点在T中有一条路径相连
- 该路径上必然存在一条边e'，其权重 ≥ w(e_k)（因为e_k是当前最小且不形成环的边）
- 将e_k加入T形成环，移除e'得到新树T'，权值 ≤ T
- 因此T'也是一棵最小生成树，且包含e_k

因此，贪心选择不会丢失最优解。✅

最优子结构：选择边e_k后，将e_k连接的两个顶点合并为一个连通分量，原问题转化为在新图上求最小生成树。

2. 伪代码（并查集实现）

text

```
算法 Kruskal(G)
// 输入：图G
// 输出：最小生成树的边集和总权重

1.  将E中所有边按权重升序排序
2.  totalWeight ← 0
3.  edgeSet ← ∅
4.  MakeSet(V)                  // 初始化并查集，每个顶点自成一个集合
5.
6.  for each edge (u, v, w) in sorted order do
7.      if Find(u) ≠ Find(v) then  // u和v属于不同连通分量
8.          totalWeight ← totalWeight + w
9.          edgeSet ← edgeSet ∪ {(u, v, w)}
10.         Union(u, v)             // 合并两个分量
11.         if |edgeSet| = |V| - 1 then
12.             break               // 已生成V-1条边，结束
13.         end if
14.     end if
15. end for
16.
17. return totalWeight, edgeSet
```

并查集操作：

- MakeSet(x)：初始化，parent[x] = x，rank[x] = 0
- Find(x)：路径压缩，返回x所在集合的代表元
- Union(x, y)：按秩合并，将两个集合合并

3. 时间复杂度分析

- 排序：O(E log E)
- 并查集操作：E次Find和V-1次Union，总复杂度接近 O(E × α(V))，其中α(V)是反阿克曼函数，增长极慢，可视为常数
- 总时间复杂度：**O(E log E)**
- 由于在连通图中 E ≥ V-1，且 log E = O(log V)，也可写作 **O(E log V)**
- 空间复杂度：**O(V + E)**

4. Prim与Kruskal对比

特性	Prim算法	Kruskal算法
贪心策略	每次选连接U和V-U的最小边	每次选全局最小且不形成环的边
数据结构	优先队列（堆）	并查集
时间复杂度	O((V+E) log V)	O(E log E)
适用场景	稠密图（E接近V²）	稀疏图（E接近V）
实现复杂度	较复杂（需维护优先队列）	较简单（排序+并查集）
起始点	需要指定起始顶点	不需要起始顶点

题目5：Dijkstra算法——单源最短路径

【问题描述】 给定一个带权有向图（或无向图）G = (V, E)，以及一个源点s ∈ V，每条边的权重w(u, v) ≥ 0。求从源点s到所有其他顶点的最短路径长度。

【输入示例】 图结构：

text

```
顶点数 n = 5
起点 s = 1
边列表（有向）：(1,2,10), (1,3,5), (2,3,2), (2,4,1), (3,2,3), (3,4,9), (3,5,2), (4,5,4), (5,4,6)
```

【输出示例】
从1出发的最短距离：
dist[1]=0, dist[2]=8, dist[3]=5, dist[4]=9, dist[5]=7
路径：1→3→2→4, 1→3→5 等

【答题框架】

1. 贪心策略分析

贪心策略（Dijkstra算法）：维护一个已确定最短路径的顶点集合S，初始只包含源点s。每次从V-S中选择**当前dist值最小**的顶点u加入S，然后松弛u的所有出边（即检查是否能通过u改善其他顶点的距离）。

贪心选择性质证明：

设顶点u是V-S中dist值最小的顶点，即dist[u] = min{ dist[v] | v ∈ V-S }。

要证明：dist[u]已经是源点s到u的最短路径长度。

- 假设存在一条更短的路径P从s到u，路径长度 < dist[u]
- 这条路径必然在某个点离开S（因为s∈S，u∉S），设离开点为x∈S，下一个点为y∈V-S
- 则P从s到y的长度 ≥ dist[y]（因y∈V-S，dist[y]是已知最短距离的上界）
- 由于所有边权非负，从y到u的路径长度 ≥ 0，所以整条路径长度 ≥ dist[y]
- 但dist[y] ≥ dist[u]（因为u是V-S中dist最小的），矛盾

因此，贪心选择不会丢失最优解。✅

最优子结构：s到u的最短路径中，s到路径上任意中间顶点的子路径也是最短路径。

2. 伪代码（优先队列优化版）

text

```
算法 Dijkstra(G, s)
// 输入：图G，源点s
// 输出：从s到所有顶点的最短距离dist和路径前驱pre

1.  for each vertex v in V do
2.      dist[v] ← ∞
3.      pre[v] ← NIL          // 记录前驱顶点，用于路径回溯
4.  end for
5.  dist[s] ← 0
6.  Q ← 优先队列（最小堆），初始包含所有顶点，按dist值排序
7.
8.  while Q is not empty do
9.      u ← ExtractMin(Q)     // 取出dist最小的顶点
10.
11.     for each neighbor v of u do
12.         // 松弛操作
13.         if dist[u] + w(u, v) < dist[v] then
14.             dist[v] ← dist[u] + w(u, v)
15.             pre[v] ← u
16.             DecreaseKey(Q, v, dist[v])   // 更新优先队列中的dist值
17.         end if
18.     end for
19. end while
20.
21. return dist, pre
```

路径回溯：通过pre数组，从终点反向追溯即可得到最短路径。

3. 时间复杂度分析

- 使用二叉堆实现优先队列：
  - ExtractMin操作：O(log V)，共V次 → O(V log V)
  - DecreaseKey操作：O(log V)，共E次 → O(E log V)
- 总时间复杂度：**O((V + E) log V)**
- 对于稠密图（$E \approx V^2$），可用斐波那契堆优化至 **O(E + V log V)**
- 空间复杂度：**O(V + E)**

4. 算法限制与对比

限制条件：Dijkstra算法要求所有边权**非负**。若存在负权边，算法会失效（因为贪心选择性质不成立）。

负权边的解决方案：使用Bellman-Ford算法（O(VE)），可处理负权边但不能有负权环。

算法	时间复杂度	适用场景	限制
Dijkstra	O((V+E) log V)	非负权图	边权 ≥ 0
Bellman-Ford	O(VE)	可处理负权边	无负权环
Floyd-Warshall	$O(V^3)$	全源最短路径	无负权环

贪心算法复杂度总结表

问题	贪心策略	时间复杂度	空间复杂度	关键数据结构
找零钱	每次取最大面额	O(k)	O(1)	无
活动选择	每次取结束最早	O(n log n)	O(1)	排序
哈夫曼编码	每次取频率最小两个	O(n log n)	O(n)	最小堆
Prim	每次取连接U的最小边	O((V+E) log V)	O(V+E)	最小堆
Kruskal	每次取全局最小不环边	O(E log E)	O(V+E)	并查集
Dijkstra	每次取dist最小顶点	O((V+E) log V)	O(V+E)	最小堆